

Email OTP verification with Redis and Brevo

How sign-up came to verify the address first — what each piece does, why it is built that way, and the two mistakes worth not repeating.

Every code block in this document is sliced out of the real source file when the document is built, so it cannot drift from what shipped.

Contents

- 1** What we are building

- 2** Why Redis rather than a table

- 3** Step 1 — the Redis connection

- 4** Step 2 — three keys, three lifetimes

- 5** Step 3 — generating and storing the code

- 6** Step 4 — verifying, and why six digits is safe

- 7** Step 5 — Brevo

- 8** Step 6 — the endpoints

- 9** Step 7 — the two-step form

- 10** Step 8 — testing it without a mailbox

- 11** The Redis commands used, and why

- 12** What to watch for

1 What we are building

Before this change, `POST /auth/register` created the account immediately. Anyone could sign up with an address they did not own — a typo, or somebody else's.

Now signing up takes two requests. A six-digit code goes to the address, and the account is only created once that code comes back. Nothing is written to Postgres in between.

1 Browser posts the form to `/auth/register/start`

↓

2 Server checks the address is free, hashes the password, generates a code, writes its keys to **Redis**

↓

3 Server hands the code to **Brevo**, which emails it

↓

4 Browser posts the code to `/auth/register/verify`

↓

5 Code matches → the `User` row is created, tokens issued, Redis keys deleted

Why no database row until step 5. An abandoned sign-up leaves nothing behind, so there is no cleanup job for half-made accounts. And because `User.email` is unique, a row created at step 1 would let anyone take a real person's address out of circulation by starting a sign-up with it and walking away. It also means no migration and no `emailVerified` column.

2 Why Redis rather than a table

The code has to disappear after sixty seconds. In Postgres that means an `expiresAt` column, a comparison against it on every read, and something running periodically to delete the dead rows — three moving parts to express one idea.

Redis expiry *is* the feature. `SET key value EX 60` and the key is simply gone a minute later. Nothing to compare, nothing to sweep up.

That is the whole reason Redis is here. In this design it is not a cache; it is a place where the deletion is automatic.

3 Step 1 — the Redis connection

One shared, lazily-opened connection. `REDIS_URL` is optional so the rest of the app runs without it:

```
backend/src/lib/env.ts
```

```
/**
 * Redis, for signup one-time codes.
 *
 * Optional so the rest of the app runs without it, but signup refuses to
 * start rather than fall back to memory: a code held in one process is
 * invisible to the next request when more than one instance is running,
 * which fails as "wrong code" on a code that was right.
 */
REDIS_URL: z.string().optional(),
```

The client itself:

```
backend/src/lib/redis.ts
```

```
export function redis(): Redis {
  if (!env.REDIS_URL) {
    throw new Error('REDIS_URL is not set, so one-time codes cannot be stored.');
```

```
  }

  if (client) return client;

  client = new Redis(env.REDIS_URL, {
    // A command must not be able to hang a request forever, but it must be
    // allowed to wait out the opening TLS handshake. Disabling the offline
    // queue does the first at the cost of the second: the very first command
    // is rejected outright because the socket is still connecting, which on a
    // cold process means the first signup of the day fails and the next one
    // works. So the queue stays on and the bounds are timeouts instead.
    maxRetriesPerRequest: 2,
    connectTimeout: 5_000,
    commandTimeout: 5_000,
    // Upstash and Fly both terminate TLS; the URL scheme decides, and ioredis
    // reads rediss:// on its own. Nothing to configure here.
    lazyConnect: false,
  });
```

Two things that bit me here

The URL scheme decides TLS. Upstash gives you a `redis-cli` command to copy. Pasting the whole thing in fails, and so does keeping the `redis://` scheme — `redis-cli` takes `--tls` as a separate flag, but a client library reads TLS from the scheme. It has to be `rediss://`, with two s characters:

```
what Upstash hands you, and what the variable needs
```

```
wrong  REDIS_URL=redis-cli --tls -u redis://default:PASSWORD@host.upstash.io:6379
wrong  REDIS_URL=redis://default:PASSWORD@host.upstash.io:6379
right  REDIS_URL=rediss://default:PASSWORD@host.upstash.io:6379
```

Turning off the offline queue broke the first request. I set `enableOfflineQueue: false` so a command could not hang forever against a dead Redis. It also rejects the very first command on a cold process, because the TLS handshake has not finished yet — so the first sign-up after every deploy would fail and the

next would work. The fix is to leave the queue on and bound it with timeouts instead, which is what the code above does.

I only found that by connecting to the real database. Reasoning about it had convinced me the option was correct.

4 Step 2 — three keys, three lifetimes

Each in-flight sign-up has keys that expire at different times, on purpose:

KEY	HOLDS	TTL
otp:<email>	the code's SHA-256 hash and the attempt count	60 seconds
signup:<email>	name, company, email, bcrypt password hash	15 minutes
otp:cooldown:<email>	nothing — its existence is the fact	30 seconds

The code is the secret, so it is the thing that must be short-lived. The form is not a secret — the person filling it in supplied every field — so it outlives several codes. That is what makes a resend cost six digits instead of the whole form again.

The cooldown key is a trick worth knowing: it stores no value. Asking `TTL otp:cooldown:someone@example.com` answers "how many seconds until they may ask for another code", and when the key expires the restriction lifts by itself.

backend/src/services/OtpService.ts

```
/** Sixty seconds, as asked for. */
const CODE_TTL_SECONDS = 60;

/** Long enough to survive a few resends without retyping the form. */
const PENDING_TTL_SECONDS = 15 * 60;

/** Wait between sends, so the endpoint cannot be used to bomb an inbox. */
const RESEND_COOLDOWN_SECONDS = 30;

/**
 * Wrong guesses before the code is burned.
 *
 * A six-digit code is a million combinations, which sounds like a lot until you
 * notice a script can try thousands inside the sixty seconds. The cap, not the
 * length, is what makes the code safe.
 */
const MAX_ATTEMPTS = 5;
```

```
backend/src/services/OtpService.ts
```

```
const codeKey = (email: string) => `otp:${normalise(email)}`;  
const pendingKey = (email: string) => `signup:${normalise(email)}`;  
const cooldownKey = (email: string) => `otp:cooldown:${normalise(email)}`;
```

5 Step 3 — generating and storing the code

Three small functions carry all the security in this file:

```
backend/src/services/OtpService.ts
```

```
/** SHA-256 is right here: the input has a million possibilities and sixty seconds of life. */  
function hash(code: string): string {  
  return crypto.createHash('sha256').update(code).digest('hex');  
}  
  
/** Six digits, uniformly distributed, from the OS. Math.random is not a secret. */  
function generateCode(): string {  
  return String(crypto.randomInt(0, 1_000_000)).padStart(6, '0');  
}  
  
/** Compared byte by byte so the time taken says nothing about how close a guess was. */  
function sameCode(a: string, b: string): boolean {  
  const left = Buffer.from(a);  
  const right = Buffer.from(b);  
  if (left.length !== right.length) return false;  
  return crypto.timingSafeEqual(left, right);  
}
```

- `crypto.randomInt`, not `Math.random`. `Math.random` is predictable from previous outputs. This is a secret, so it comes from the operating system.
- **Stored as a hash.** Anyone reading the Redis database — a leaked URL, a support tool, a backup — sees a hash, not a code they could use.
- **Compared with `timingSafeEqual`.** A normal `===` on strings returns as soon as two bytes differ, so how long it takes leaks how much of the guess was right.

Writing it happens in one `MULTI`, so the keys are set together rather than a code existing for a moment with no form behind it:

backend/src/services/OtpService.ts

```
static async issue(pending: PendingSignup): Promise<SendDecision> {
  const client = redis();
  const email = normalise(pending.email);

  const cooling = await client.ttl(cooldownKey(email));
  if (cooling > 0) return { ok: false, retryInSeconds: cooling };

  const code = generateCode();
  const record: CodeRecord = { hash: hash(code), attempts: 0 };

  await client
    .multi()
    .set(codeKey(email), JSON.stringify(record), 'EX', CODE_TTL_SECONDS)
    .set(pendingKey(email), JSON.stringify({ ...pending, email }), 'EX', PENDING_TTL_SECONDS)
    .set(cooldownKey(email), '1', 'EX', RESEND_COOLDOWN_SECONDS)
    .exec();

  return { ok: true, expiresInSeconds: CODE_TTL_SECONDS, code };
}
```

Notice what `issue` returns. It hands the code back to its caller rather than sending the email itself. That is what lets the test suite read a real code without a mailbox, and it keeps this file free of any knowledge of Brevo.

6 Step 4 — verifying, and why six digits is safe

Six digits is one million combinations. That sounds strong until you notice a script can try many thousands inside the sixty seconds the code is alive. **The attempt cap, not the length, is what makes it safe.**

```

static async verify(email: string, code: string): Promise<VerifyResult> {
  const client = redis();
  const key = codeKey(email);

  const raw = await client.get(key);
  if (!raw) return { ok: false, reason: 'expired', attemptsLeft: 0 };

  let record: CodeRecord;
  try {
    record = JSON.parse(raw) as CodeRecord;
  } catch {
    await client.del(key);
    return { ok: false, reason: 'expired', attemptsLeft: 0 };
  }

  if (!sameCode(record.hash, hash(code))) {
    const attempts = record.attempts + 1;

    if (attempts >= MAX_ATTEMPTS) {
      // Burn the code rather than the sign-up: the form survives, so they can
      // ask for a new code without filling it in again.
      await client.del(key);
      return { ok: false, reason: 'exhausted', attemptsLeft: 0 };
    }

    // Keep whatever life the code had left. Re-setting the TTL here would let
    // a wrong guess extend the window it is trying to guess inside.
    const ttl = await client.ttl(key);
    const next: CodeRecord = { ...record, attempts };
    if (ttl > 0) await client.set(key, JSON.stringify(next), 'EX', ttl);
    else await client.del(key);

    return { ok: false, reason: 'wrong', attemptsLeft: MAX_ATTEMPTS - attempts };
  }

  const pending = await this.pending(email);
  if (!pending) return { ok: false, reason: 'expired', attemptsLeft: 0 };

  await client.del(key, pendingKey(email), cooldownKey(email));
  return { ok: true, pending };
}

```

Three decisions in there worth pulling out:

- **Five wrong guesses burn the code, not the sign-up.** The `signup:` key survives, so the user asks for a new code rather than refilling the form.
- **A wrong guess must not reset the TTL.** The code reads the remaining time and writes it back. Calling `SET ... EX 60` here would let an attacker extend the very window they are guessing inside, simply by guessing.
- **A correct code is deleted.** Without that, a repeated request — a double-click, a retry — could create two accounts from one code.

7 Step 5 — Brevo

Brevo is a plain HTTPS API. No SDK is involved: one `fetch` to `/v3/smtp/email` with the key in an `api-key` header.

backend/src/lib/email/EmailService.ts

```
const res = await fetch('https://api.brevo.com/v3/smtp/email', {
  method: 'POST',
  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json',
    'api-key': env.API_KEY_FOR_EMAIL,
  },
  body: JSON.stringify({
    sender: { name: env.EMAIL_FROM_NAME, email: env.EMAIL_FROM },
    to: [{ email: to, name: toName ?? to }],
    subject,
    htmlContent: html,
    textContent: text,
  }),
});

if (!res.ok) {
  throw new Error(`Brevo rejected the message (${res.status}): ${await res.text()}`);
}
```

The failure mode that makes Brevo worth guarding

Brevo answers **201 Created** and *then* silently drops the message if the sender address is not one your account has verified. The send looks successful, the mail never arrives, and nothing in any log says so.

So the app asks Brevo at boot which senders are verified, and refuses to send from an address that is not among them:

backend/src/lib/email/EmailService.ts

```
export async function verifyEmailSender(): Promise<void> {
  if (!env.API_KEY_FOR_EMAIL) {
    console.log('[email] no API key configured – emails will be logged, not sent');
    return;
  }

  try {
    const res = await fetch('https://api.brevo.com/v3/senders', {
      headers: { Accept: 'application/json', 'api-key': env.API_KEY_FOR_EMAIL },
      signal: AbortSignal.timeout(10_000),
    });

    if (!res.ok) {
      console.warn('[email] could not list senders (${res.status}); sending will be attempted unchecked');
      return;
    }

    const data = (await res.json()) as { senders?: Array<{ email: string; active: boolean }> };
    verifiedSenders = new Set((data.senders ?? []).filter((s) => s.active).map((s) => s.email.toLowerCase()));
  }
}
```

That is what the `[email]` sending as `... (verified)` line at startup means, and why `/health` reports `emailSender`.

The OTP message itself

backend/src/lib/email/EmailService.ts

```
async sendSignupOtp(args: { to: string; name?: string | null; code: string; expiresInSeconds: number }) {
  const minutes = Math.round(args.expiresInSeconds / 60);
  const validFor = args.expiresInSeconds < 90 ? `${args.expiresInSeconds} seconds` : `${minutes} minutes`;

  const body = `
    <p>Hi${args.name ? ` ${args.name}` : ''},</p>
    <p>Use this code to finish creating your account.</p>
    <div style="margin:24px 0;padding:20px;background:#f8fafc;border:1px solid #e2e8f0;border-radius:8px;text-align:center;">
      <div style="font-size:34px;font-weight:600;letter-spacing:0.22em;color:#0f172a;font-family:ui-monospace,SFMono-Regular,Menlo,monospace;">${args.code}</div>
      <div style="font-size:12px;color:#64748b;margin-top:8px;">Valid for ${validFor}</div>
    </div>
    <p style="color:#64748b;font-size:14px;">If you did not try to sign up, you can ignore this – no account has been created, and nothing happens unless the code is used.</p>`;

  await this.send({
    to: args.to,
    toName: args.name ?? undefined,
    subject: `${args.code} is your verification code`,
    html: shell('Verify your email', body),
  });
}
```

No links, deliberately. A verification email whose only instruction is "type these six digits into the tab you already have open" cannot be repurposed as a phishing template. Anything clickable in an email teaches people to click things in emails.

8 Step 6 — the endpoints

Three routes replace the single old one:

backend/src/routes/index.ts

```
// Signing up is two steps: a code to the address, then the account.
router.post('/auth/register/start', asyncHandler(auth.registerStart));
router.post('/auth/register/verify', asyncHandler(auth.registerVerify));
router.post('/auth/register/resend', asyncHandler(auth.registerResend));
```

Step one — validate, hash, issue, send. The password is hashed *before* it goes into Redis, so it is never sitting there in the clear even for the minutes it waits:

backend/src/controllers/auth.controller.ts

```
export const registerStart = async (req: AuthRequest, res: Response) => {
  requireOtpStore();
  const data = registerSchema.parse(req.body);

  const existing = await prisma.user.findUnique({ where: { email: data.email } });
  if (existing) throw conflict('An account with that email already exists');

  const issued = await OtpService.issue({
    email: data.email,
    // Hashed before it is stored, so a password never sits in Redis in the clear
    // even for the minutes this waits.
    passwordHash: await bcrypt.hash(data.password, 10),
    name: data.name ?? null,
    company: data.company ?? null,
  });

  if (!issued.ok) {
    throw badRequest(`A code was just sent. Try again in ${issued.retryInSeconds} seconds.`);
  }

  await emailService.sendSignupOtp({
    to: data.email,
    name: data.name ?? null,
    code: issued.code,
    expiresInSeconds: issued.expiresInSeconds,
  });

  res.status(202).json({
    email: data.email,
    expiresInSeconds: issued.expiresInSeconds,
    resendInSeconds: OtpService.resendCooldownSeconds,
  });
};
```

Step two — the code is right, so the account becomes real:

```

export const registerVerify = async (req: AuthRequest, res: Response) => {
  requireOtpStore();
  const { email, code } = verifySchema.parse(req.body);

  const result = await OtpService.verify(email, code);

  if (!result.ok) {
    if (result.reason === 'expired') {
      throw badRequest('That code has expired. Ask for a new one.');
```

```

    }
    if (result.reason === 'exhausted') {
      throw badRequest('Too many wrong attempts. Ask for a new code.');
```

```

    }
    throw badRequest(
      `That code is not right. ${result.attemptsLeft} attempt${result.attemptsLeft === 1 ? '' : 's'} left.`,
    );
  }
}
```

```

const pending = result.pending;
```

```

// Checked again here, not only at step one: minutes passed in between, and
// two people can hold a code for the same address at once. The unique index
// is the real guard, this is the readable error.
```

```

const existing = await prisma.user.findUnique({ where: { email: pending.email } });
if (existing) throw conflict('An account with that email already exists');
```

```

const user = await prisma.user.create({
  data: {
    email: pending.email,
    password: pending.passwordHash,
    name: pending.name,
    company: pending.company,
  },
});
```

```

const payload: AuthUser = { userId: user.id, email: user.email, role: user.role };
setRefreshCookie(res, signRefreshToken(payload));
```

```

res.status(201).json({ accessToken: signAccessToken(payload), user: publicUser(user) });
};
```

The address is checked for availability *twice* — once at step one and again at step two. Minutes pass in between, and two people can hold a code for the same address at once. The unique index on `User.email` is the real guard; this second check is what turns a database constraint violation into a sentence a person can read.

9 Step 7 — the two-step form

The register page keeps both steps in one component and holds the details in state, so going back to fix a typo in the address does not mean retyping the password.

```
frontend-v2/src/app/(auth)/register/page.tsx
```

```
const [step, setStep] = useState<'details' | 'code'>('details');
const [form, setForm] = useState({ name: '', company: '', email: '', password: '', confirm: '' });
const [code, setCode] = useState('');
const [error, setError] = useState('');
const [notice, setNotice] = useState('');
const [submitting, setSubmitting] = useState(false);

/** Seconds until the code dies, and until another can be sent. */
const [expiresIn, setExpiresIn] = useState(0);
const [resendIn, setResendIn] = useState(0);

const codeInput = useRef<HTMLInputElement>(null);
```

One interval drives both countdowns — the remaining life of the code, and the wait until another can be sent:

```
frontend-v2/src/app/(auth)/register/page.tsx
```

```
// One interval drives both countdowns; they only ever tick down.
useEffect(() => {
  if (step !== 'code') return;

  const id = setInterval(() => {
    setExpiresIn((s) => (s > 0 ? s - 1 : 0));
    setResendIn((s) => (s > 0 ? s - 1 : 0));
  }, 1000);

  return () => clearInterval(id);
}, [step]);
```

The input is filtered to digits as it is typed, and carries `autoComplete="one-time-code"` so phones offer the code from the notification:

```
frontend-v2/src/app/(auth)/register/page.tsx
```

```
onChange={(e) => setCode(e.target.value.replace(/\D/g, '').slice(0, 6))}
placeholder="123456"
inputMode="numeric"
autoComplete="one-time-code"
```

10 Step 8 — testing it without a mailbox

`npm run verify:otp` runs against the real Redis in `.env` and sends no email, because `issue()` returns the code to its caller.

The tests worth having are the failure modes, not the happy path:

backend/scripts/otptest.ts

```
console.log('3. The attempt cap is what makes six digits safe');

for (let i = 0; i < 3; i++) await OtpService.verify(email, '000001');
const fifth = await OtpService.verify(email, '000001');
check(
  'the code is burned after five wrong attempts',
  !fifth.ok && fifth.reason === 'exhausted',
  !fifth.ok ? fifth.reason : 'accepted',
);

const afterBurn = await OtpService.verify(email, issued.code);
check('and the right code no longer works either', !afterBurn.ok);

const survived = await OtpService.pending(email);
check('but the form survives, so a resend costs six digits not the whole form', survived !== null);
```

Expiry is tested by shortening the TTL rather than waiting sixty seconds. The expiry under test is Redis's own, and it does not care how the TTL got short:

backend/scripts/otptest.ts

```
// The real code lives 60s. Waiting that out would make this suite unusable,
// so the key is aged directly – the expiry being tested is Redis's, and it
// does not care how the TTL got short.
const fresh = `otptest+exp${Date.now()}@example.com`;
const short = await OtpService.issue({ ...signup, email: fresh });
if (short.ok) {
  await redis().expire(`otp:${fresh}`, 1);
  await wait(1600);
  const expired = await OtpService.verify(fresh, short.code);
  check('an expired code is refused', !expired.ok && expired.reason === 'expired');
```

npm run verify:otp

```
1. Issuing a code
PASS the code is six digits - 053497
PASS the code is not stored in the clear
PASS the form is held alongside it
2. Wrong codes
PASS a wrong code is refused and counted - 4 left
PASS a wrong guess does not extend the code - 60s left of 60
3. The attempt cap is what makes six digits safe
PASS the code is burned after five wrong attempts
PASS but the form survives, so a resend costs six digits not the whole form
4. Resend
PASS a resend inside the cooldown is refused - 29s
5. The right code
PASS a used code cannot be used twice
6. Expiry
PASS an expired code is refused

OTP OK
```

11 The Redis commands used, and why

COMMAND	USED FOR
<code>SET key value EX 60</code>	Write with a lifetime in one step. The <code>EX</code> is the whole reason Redis is here.
<code>GET key</code>	Read the code record or the pending form. A missing key <i>is</i> expiry — there is no date to compare.
<code>TTL key</code>	Two jobs: how long the cooldown has left, and how much life to preserve when writing back an incremented attempt count.
<code>MULTI ... EXEC</code>	Set the code, the form and the cooldown together, so a code never exists without a form behind it.
<code>DEL k1 k2 k3</code>	Consume everything on success, so one code cannot make two accounts.
<code>EXPIRE key 1</code>	Test only — ages a key so expiry can be checked without waiting a minute.

12 What to watch for

- **Sixty seconds is tight.** Brevo usually delivers in a few seconds, but a twenty-second delivery leaves forty seconds to switch apps, find the mail and type. Expect real users to need the resend button; it is a support cost, not a hypothetical.
- **No in-memory fallback, on purpose.** A code held in one process is invisible to every other process, so with more than one instance running, a *correct* code entered against the wrong instance reads as wrong. That looks like a bug in the code generator and is miserable to trace. Better to refuse to start and say why.
- **The resend endpoint says the same thing either way.** Whether or not an address has a sign-up waiting, the response is identical — otherwise it becomes a way to find out who has started signing up.
- **Deployment.** `fly secrets set REDIS_URL=...` before this works in production. Without it, sign-up refuses cleanly rather than half-working.